

VoidShare

Peer-to-Peer Encrypted File Sharing

Version 2.0 — Project Report

Key-exchange race-condition fix, full project restructuring, and a redesigned interface.

Repository: github.com/Luv-valecha/VoidShare

Report generated: September 6, 2026

This report documents the changes made in VoidShare V2.0: the root cause and fix for the DataChannel send-time crash, the reorganized **server/** + **client/** project layout, the componentized front-end architecture, and the visual redesign of the UI. It is intended as a technical changelog and reference for anyone picking up the codebase after this release.

Contents

1.	Executive Summary	3
2.	The Bug: Root Cause Analysis	4
3.	The Fix	5
4.	Project Restructuring	6
5.	Front-End Architecture (Before / After)	7
6.	UI / UX Redesign	8
7.	Security Model (Unchanged Guarantees, Hardened Delivery)	9
8.	Dependency & Security Maintenance	10
9.	How to Run V2.0	10
10.	Changelog Summary	11

1. Executive Summary

VoidShare is a privacy-first, peer-to-peer file sharing application. Files travel directly between two browsers over a WebRTC DataChannel, encrypted end-to-end with AES-256-GCM and an ECDH (P-256) key exchange; a small signaling server only helps the two peers find each other and never sees file contents.

This release (V2.0) was triggered by a reported runtime crash when sending a file:

```
Error in handleFileSend: TypeError: Failed to execute 'deriveKey' on
'SubtleCrypto': EcdhKeyDeriveParams: public: Must be a CryptoKey
```

Beyond fixing that crash, the project was reorganized end-to-end and the interface was given a visual refresh. The table below summarizes the scope of this release.

Area	V1.0	V2.0
Bug	Send crashes intermittently with a CryptoKey TypeError	Root cause fixed; send is disabled until the secure channel is verified ready
Layout	Mixed root: server.js at repo root, frontend/ folder, monolithic page.js	Clean server/ + client/ split; components, hooks, and lib separated
page.js	~590 lines, all state, all markup, all handlers in one file	~65 lines; composes small, focused components + one hook
UI	Functional dark theme, emoji icons, no visibility into connection internals	Glass-panel cards, ambient background, lucide icons, live status pills
Dependencies	Next.js 15.3.4 (has a disclosed critical RCE, CVE-2025-66478)	Next.js 15.5.25 (latest patched release on the 15.x line)

2. The Bug: Root Cause Analysis

2.1 Symptom

A connection between two peers would establish successfully (the UI showed “Connected to <peer>”), but selecting a file and pressing **Send File** would throw immediately, with no file data ever leaving the browser.

2.2 Where it happened

In the original frontend/app/page.js, handleFileSend derived a shared AES key like this:

```
const sharedKey = await deriveSharedKey(privateKey, peerPublicKey);
// crypto_utils.js
export const deriveSharedKey = async (privateKey, publicKey) => {
  return crypto.subtle.deriveKey(
    { name: "ECDH", public: publicKey },
    privateKey,
    { name: "AES-GCM", length: 256 },
    true,
    ["encrypt", "decrypt"]
  );
};
```

crypto.subtle.deriveKey requires its public parameter to be an already-imported CryptoKey. If peerPublicKey is still null at call time, the browser throws exactly the reported error.

2.3 Why peerPublicKey was still null

The original key-exchange handshake was **asymmetric**:

- The **calling** peer (the one who clicked Connect) sent its own public ECDH key as soon as its DataChannel's onopen fired.
- The **answering** peer (the one who clicked Accept) did *not* send its key proactively. It only sent a key back reactively, inside a useEffect that fired once it had already received the other side's key.
- Meanwhile, the connected React state — which is what the UI used to enable the file picker and Send button — was set from the signaling server's “accepted” / “decline” event. That event is purely about the WebSocket handshake and has no relationship to whether the DataChannel key exchange has actually finished.

Put together: a user could see “**Connected**” and have a fully open DataChannel well before peerPublicKey arrived. Clicking Send File in that window — which, on a fast local network, could be a window of well under a second — called deriveKey with a null public key and crashed. This also explains why the bug was intermittent: on a slow connection or when the user paused before selecting a file, the reactive exchange usually finished in time by coincidence.

3. The Fix

3.1 Symmetric, eager key exchange

In `client/src/lib/signaling.js`, both the calling side and the answering side now attach the exact same `onChannelOpen` handler to their `DataChannel`. Whichever side's channel opens, it immediately exports and sends its own public ECDH key — independent of anything the other side has done:

```
function attachDataChannel(channel, handlers) {
  dataChannel = channel;
  dataChannel.onopen = () => handlers.onChannelOpen?();
  dataChannel.onmessage = (e) => handlers.onData?.(e.data);
  // ...
}

// useVoidShare.js — the SAME handler is used for both initiator and acceptor:
onChannelOpen: () => {
  setChannelOpen(true);
  sendPublicKey(); // <-- fired on BOTH sides now, not just the caller
}
```

This removes the ordering dependency entirely: neither side waits on the other before sending its key, so there is no longer a window where one side has sent but not the other.

3.2 A dedicated “secure channel ready” state

Rather than gating the Send button on the signaling-level connected flag, V2.0 introduces `keysReady`:

```
const keysReady = Boolean(keyPairRef.current.privateKey && peerPublicKey);
```

The Send File button is disabled (and shows “Waiting for secure channel...”) until `keysReady` is true, and the connection panel shows three independent, honestly-labeled status pills: **Connected**, **Channel open**, and **Secure channel ready**. `handleFileSend` and `handleDownload` also both check `keysReady` defensively and surface a friendly toast instead of throwing if called too early.

3.3 Defensive check inside the crypto helper itself

As a second line of defense, `deriveSharedKey` in `client/src/lib/crypto.js` now checks its inputs before calling into Web Crypto, so a future caller anywhere in the codebase gets a clear, descriptive error instead of a cryptic `DOMException`:

```
export async function deriveSharedKey(privateKey, publicKey) {
  if (!privateKey || !publicKey) {
    throw new Error(
      "Cannot derive shared key: the ECDH key exchange with your peer " +
      "hasn't finished yet."
    );
  }
  return crypto.subtle.deriveKey(/* ... */);
}
```

3.4 Related hardening: clean connection teardown

The original module-level WebRTC state (`localConnection`, `dataChannel`, etc.) was never reset on disconnect. V2.0 adds an explicit `closeConnection()` that closes the `DataChannel` and `RTCPeerConnection` and clears all module state, called from `disconnectPeer()`, so reconnecting after a disconnect starts from a clean slate instead of stale references.

4. Project Restructuring

The repository previously mixed the backend (`server.js`, `root package.json`) with the frontend (`frontend/`) at the top level, and most of the front-end logic lived in a single ~590-line `frontend/app/page.js`. V2.0 splits the project into two independent, independently-deployable packages:

```
voidshare/
|-- docs/                V2.0 report + architecture diagram
|-- docker-compose.yml
|-- server/              Signaling server (Express + ws)
|   |-- index.js
|   |-- package.json
|   |-- Dockerfile
|-- client/              Next.js 15 app
|   |-- src/
|       |-- app/
|           |-- page.js    thin composition (~65 lines)
|           |-- layout.js
|           |-- globals.css
|           |-- devlogs/page.js
|       |-- components/    Header, PeerIdCard, ConnectPanel,
|                           FileTransferPanel, ProgressBar,
|                           QrCode, LoadingScreen
|       |-- hooks/
|           |-- useVoidShare.js    connection + key exchange + transfer
|           |-- useBackendWarmup.js
|       |-- lib/
|           |-- signaling.js    WebRTC + WS (bug-fixed)
|           |-- crypto.js       AES-GCM + ECDH helpers
|-- package.json
|-- Dockerfile
```

Notable renames and moves:

V1.0 path	V2.0 path	Why
<code>server.js</code> (repo root)	<code>server/index.js</code>	Backend is now a self-contained package
<code>frontend/</code>	<code>client/</code>	Symmetric naming with <code>server/</code>
<code>frontend/lib/server_utils.js</code>	<code>client/src/lib/signaling.js</code>	Name reflects what it does; bug-fixed
<code>frontend/lib/crypto_utils.js</code>	<code>client/src/lib/crypto.js</code>	Consistent naming; added guard clauses
<code>frontend/app/page.js</code> (590 lines)	<code>src/app/page.js</code> (~65 lines) + <code>hooks/</code> + <code>components/</code>	Single Responsibility; testable pieces
<code>frontend/components/QrCode.js</code>	<code>src/components/QrCode.js</code>	Restyled, unchanged behavior

5. Front-End Architecture (Before / After)

5.1 Before

A single client component owned: all connection state, the ECDH key pair, the signaling callbacks, the file-send pipeline, the file-receive pipeline, progress tracking, and the entire JSX tree for every panel on the page. Every change — UI or logic — touched the same file.

5.2 After

Logic and presentation are now separated along clear boundaries:

Layer	Responsibility
lib/signaling.js	Owns the WebSocket + RTCPeerConnection/DataChannel lifecycle. Pure functions, no React.
lib/crypto.js	Owns all Web Crypto calls (ECDH, AES-GCM, hashing). Pure functions, no React.
hooks/useVoidShare.js	Orchestrates the above into React state: connection status, keysReady, transfer progress, and the handlers the UI calls.
hooks/useBackendWarmup.js	Isolated concern: wake a sleeping signaling host before connecting.
components/*	Presentation only — each receives plain props and renders one panel (peer ID, connect form, transfer controls, QR code, status pills).
app/page.js	Composition root: wires the hook's state/handlers into the components. No business logic.

This makes the fix in Section 3 possible to reason about in isolation — the bug and its fix live entirely in `lib/signaling.js` and `hooks/useVoidShare.js`, with zero changes required to any presentational component.

6. UI / UX Redesign

The dark, red-accented brand identity is kept, but the interface has been refreshed for clarity and polish:

- **Ambient background:** two soft, slowly-drifting radial-gradient blobs (red + blue) behind the content, replacing the flat gradient background.
- **Glass-panel cards:** every section (Peer ID, Connect, Transfer, QR) sits in a translucent, blurred card with a subtle border, instead of plain colored boxes.
- **Live status pills:** the connect panel now shows real-time *Connected*, *Channel open*, and *Secure channel ready* indicators with spinners while pending — turning the previous invisible race condition into something the user can actually see resolve.
- **Icon system:** emoji (lightning bolt, scroll, plug) replaced with a consistent `lucide-react` icon set throughout headers, buttons, and status indicators.
- **File picker:** the raw HTML file input is now wrapped in a styled dashed-border drop-style control that shows the selected filename.
- **DevLogs page:** restyled as a timeline with connecting rail and glass cards per entry; a new entry documents this release's fix (Section 3) in user-facing language.

7. Security Model

The underlying cryptographic design is unchanged from V1.0 and remains sound; V2.0 hardens how reliably that design is actually reached at runtime, and fixes a resource-cleanup gap on disconnect.

- WebRTC DataChannels are DTLS-encrypted in transit by the browser itself.
- Each browser session generates an ephemeral ECDH (P-256) key pair; private keys never leave the browser.
- Every file transfer uses a fresh, random one-time AES-256-GCM key.
- That one-time file key is encrypted with the ECDH-derived shared secret before being sent, alongside a SHA-256 hash of the plaintext for integrity verification on receipt.
- The signaling server sees only SDP/ICE handshake metadata — never file bytes, keys, or the shared secret.

What changed: the reliability of the handshake that establishes the shared secret (Section 3), and proper teardown of the `RTCPeerConnection/DataChannel` on disconnect so stale WebRTC state cannot leak into a subsequent session.

8. Dependency & Security Maintenance

While rebuilding the client, `npm install` flagged that Next.js 15.3.4 (the version pinned in V1.0) carries a disclosed critical vulnerability, CVE-2025-66478 (React Server Components RCE, CVSS 10.0), along with two follow-up advisories (CVE-2025-55184, CVE-2025-55183) affecting the wider 15.x line. The client now pins Next.js to **15.5.25**, the latest patched release on the 15.x maintenance line at time of writing, with matching `eslint-config-next`.

A full production build (`next build`) was run against the restructured client to confirm the new file layout, hooks, and components compile and prerender cleanly.

9. How to Run V2.0

```
# Signaling server
cd server && npm install && npm run dev

# Client (separate terminal)
cd client && npm install
export NEXT_PUBLIC_SIGNALING_SERVER_URL=ws://localhost:4000
npm run dev

# ...or with Docker Compose:
docker compose up --build
```

Full configuration, environment variables, and the signaling protocol reference are documented in the top-level `README.md`.

10. Changelog Summary

Fixed

- Send File crashing with `EcdhKeyDeriveParams: public: Must be a CryptoKey` when a file was sent before the peer's public key had arrived.
- Both peers now send their public ECDH key immediately on `DataChannel` open, removing the asymmetric handshake that caused the race.
- Stale WebRTC state after disconnect — `closeConnection()` now properly tears down the `RTCPeerConnection/DataChannel`.
- `Next.js` pinned to a version with a disclosed critical RCE (CVE-2025-66478 and follow-ups); upgraded to the latest patched 15.x release.

Added

- A `keysReady` state and three-part live status display (`Connected / Channel open / Secure channel ready`).
- Friendly, non-crashing error toasts if `send/download` is attempted before the secure channel is ready.
- A defensive input check inside `deriveSharedKey`.
- A new `DevLogs` entry documenting this fix and the restructuring.

Changed

- Repository restructured into independent `server/` and `client/` packages, each with its own `package.json` and `Dockerfile`.
- `frontend/app/page.js` (≈ 590 lines) decomposed into a ~ 65 -line composition root, two hooks, and seven presentational components.
- `server_utils.js` $\rightarrow$ `lib/signaling.js`; `crypto_utils.js` $\rightarrow$ `lib/crypto.js`, both documented and cleaned up.
- Visual redesign: glass-panel cards, ambient background, lucide-react iconography, live status pills, restyled `DevLogs` timeline.
- `docker-compose.yml` and both `Dockerfiles` updated for the new `server/ + client/` layout.